**Student Name (in English):** _____ **Student ID:** _____

**Question 1:** *True/False* To be answered in **(6) Minutes** [ ] / 2 Marks

For each of the statements below, determine whether it is **True** or **False**. Write one answer clearly.

| Statement | True or False |
|---|---|
| In asymptotic analysis, constants and lower-order terms are ignored. | |
| Recurrence relations always result in exponential solutions. | |
| Merge Sort uses a divide-and-conquer strategy to sort an array. | |
| Insertion Sort is more efficient than Merge Sort for sorting large datasets. | |

**Question 2:** *MCQs* To be answered in **(12) Minutes** [ ] / 4 Marks

**YOU MUST USE THE PROVIDED TABLE BELOW. USE CAPITAL LETTERS (A, B, C, D)**

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| | | | |

1. **Which of the following Big-O notations represents the best (most efficient) time complexity?**
   A. $O(n^2)$
   B. $O(\log(n!))$
   C. $O(2^{\log n})$
   D. $O(n \log n)$

2. **Arrange the following functions in the correct order of growth (slowest to fastest):**
   A. $O(2^n), O(\sqrt{logn}), O(nlogn), O(n^2)$
   B. $O(\sqrt{logn}), O(nlogn), O(n^2), O(2^n)$
   C. $O(nlogn), O(\sqrt{logn}), O(n^2), O(2^n)$
   D. $O(2^n), O(nlogn), O(n^2), O(\sqrt{logn})$

3. **Which of the following sorting algorithms does not work in-place?**
   A. Merge Sort
   B. Insertion Sort
   C. Both
   D. Neither

Student Name (in English): _____ Student ID: _____

4. For a recurrence $T(n) = 2T(n/2) + n$, which of the following statements is true?
   A. $T(n) = \omega(n^2 logn)$
   B. $T(n) = \omega(nlogn)$
   C. $T(n) = \omega(n)$
   D. $T(n) = \omega(n^2)$

**Question 3:** *Asymptotic Notation* To be answered in **(18) Minutes** [ ] / 6 M

A. Prove that $f(n) = 3n^2 - n$ is $\theta(n^2)$ and find the constants $c_1$, $c_2$, and $n_0$. [2 marks]

B. Find the Big-O of $f(n) = (5n^4 + 7n^3 - 1)(3n^3 - 4n^2 + 8)$. [2 marks]

C. A computer can perform $10^7$ operations per second. An algorithm requires $n^2$ operations to solve a problem. Determine the largest input size $n$ that the computer can handle in **2 hours**. [2 marks]

Student Name (in English): _____ Student ID: _____

**Question 4:** *Recurrence* To be answered in **(24) Minutes** [ ] / 8 Marks

**Part 1:**

Given the following recurrence:

$$T(n) = 3\,T(n/2) + n$$

A. Draw the recursion tree based on the above recurrence. [2 marks]

B. Determine the number of levels in the tree. [1 mark]

C. Calculate the total cost (*Big-O*) of *T(n)* of all levels. [1 mark]

Student Name (in English): _____ Student ID: _____

**Part 2:**

Complete the table below by identifying which case of Master Theorem applies for each recurrence and determining its time complexity $\theta$: [4 marks]

| $T(n)$ | Case | Time complexity $\theta$ |
|---|---|---|
| $T(n) = 7T(n\backslash 2) + n^2 logn$ | | |
| $T(n) = 3T(n\backslash 4) + n^2 + n^3$ | | |
| $T(n) = 3T(n\backslash 3) + \frac{n^2}{(logn)^2}$ | | |
| $T(n) = 9T(n\backslash 3) + n^2$ | | |

**Question 5:** *Sorting* To be answered in **(15) Minutes** [ ] / 5 Marks

**Part 1:**

Consider the following array:

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| Value | 7 | 11 | 3 | 10 | 17 | 23 | 1 | 4 | 21 | 5 |

A. Consider using insertion sort, write the content of the array after three compete passes using descending order. [1 mark]

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| Value | | | | | | | | | | |

B. Give the time complexity of insertion sort in the best-case scenario and provide an example of an array of size 5 that results in this complexity. [1 mark]

Insertion sort best-case time complexity: _____

Example: _____

**Student Name (in English):** _____ **Student ID:** _____

**Part 2:**

Sorting algorithms are essential for organizing data efficiently. Merge Sort is a powerful algorithm with $O(nlogn)$ complexity, but it has overhead for small arrays. Insertion Sort, though $O(n^2)$, is faster for small inputs due to its simplicity. By combining these two, we can optimize sorting performance.

```
Modified-Merge-Sort (A, p, r): // A is the array, p is the start index of A, r is the end index of A
if p < r:
    if (1) _____ //Fill in the condition to switch to Insertion Sort
        Insertion_Sort (A, p, r)
    else
        q = (2) _____ //Fill in the middle index calculation
        Modified-Merge-Sort (A, p, q)
        Modified-Merge-Sort (A, (3)_____, (4)_____) //Fill in the range
        (5) _____ // Fill in the function to combine sorted halves
```

A. Complete the pseudocode by filling in the below table for the **Modified-Merge-Sort** algorithm that switches to **Insertion Sort** when the size of a subarray is ≤10. [2.5 marks]

| Blank | Answer |
|---|---|
| (1) | |
| (2) | |
| (3) | |
| (4) | |
| (5) | |

B. What is the overall time complexity of the **Modified-Merge-Sort**? [.5 mark]

1. $O(n^2)$
2. $O(nlog(n))$
3. $O(nlog\, n + k^2)$, *where k is the size of small subarrays*
4. $O(nlog(k))$, *where* $k \leq 10$

**Answer:** _____

*End of Questions*
*Good Luck*